SJ INNOVATION · ASSESSMENT 2 (CONTINUATION OF ASSESSMENT 1)

QuickBite — Level 2: make it AI-powered & tested

Food Ordering & Delivery · take your v1 to a production-minded, AI-driven app

Intern	Sanket Rotangar (Goa)
Mentor	Amol — Tech Manager
Issued	Friday, 25 September 2026
Due	**Wednesday, 30 September 2026** — submit online on **Dev Control Tower**
Your v1	https://quickbite-nine-phi.vercel.app/

What you built in Assessment 1

A food-delivery app that went **beyond the brief**: five roles (**Admin, Restaurant Owner, Restaurant Manager, Rider, Customer**), multi-branch restaurants, manager invites and a full order lifecycle. Deployed with Supabase + RLS.

Your project	Link
Live app	`https://quickbite-nine-phi.vercel.app/`
GitHub	`github.com/sanketrotangar-sji/quickbite-food-delivery-app`
Supabase ref	`motqehtswgjbbvoazarh`

You already exceeded Assessment 1 — this assessment is your chance to go deep on the AI + testing tracks.

Assessment 2 in one line: keep the same app, and add an **AI agent**, **2 automations**, a **RAG system on 5,000+ records**, a real **LLM integration with a ≥70%-accuracy result**, and **automated tests** — all with great UI/UX.

Your seven tracks

1 AI Agent — add new or enhance existing — 20 pts

Option A (new): A **food + support assistant**: recommends dishes ("something spicy under ₹300") and answers order-support questions.

Option B (enhance): Upgrade it to a **multi-step agent with tools** that can *place an order*, *check delivery status* and *escalate a complaint* by calling your own functions.

Do at least one well. Use Gemini (free) or Ollama for the model.

2 Two new workflows / automations — 15 pts

Smart rider assignment: when an order is ready, automatically assign the nearest / least-busy available rider.

Kitchen-load & delay alerts: detect when a branch is overloaded and automatically notify affected customers of a longer ETA.

"Workflow/automation" = something that runs on an event or schedule without a user clicking through every step.

3 RAG on ≥5,000 records — 20 pts

Menu items + reviews + order history across branches. Seed **≥5,000 records**. Ground the assistant's answers in this data.

Evaluate three query sets and report results: (a) **matching**, (b) **extreme/edge**, (c) **noisy** data (typos + junk). Report retrieval quality **with vs without noise**.

4 LLM integration + train & show ≥70% accuracy — 15 pts

Wire in a free LLM (Gemini API key, or Ollama locally). Then the classification task:

Predict a dish's **cuisine / tag** from its name+description, or **order-cancellation risk**. Report ≥70% accuracy on a held-out test set.

Split train/test, report accuracy on the held-out set, and include the number as evidence.

5 Test automation for all modules — 15 pts

Add **unit tests for every module** (auth, each role's core actions, the workflows, the RAG/agent layer) and at least one **end-to-end happy-path** test. Tests must run with a single command and pass.

6 Best-possible UI/UX — 10 pts

Polish the whole experience: loading/empty/error states, responsive layout, a clean agent chat UI, and clear feedback when automations fire.

7 Docs & Dev Control Tower submission — 5 pts

Update the README, record your RAG + accuracy metrics, and submit everything on **Dev Control Tower**.

Stretch / bonus: You are ahead of the group — aim for the bonus: run the LLM **locally via Ollama** and have the agent complete a real order end-to-end.

Tooling & how-to (free options)

LLM & API keys (free)

- **Google Gemini** — get a free API key at `aistudio.google.com` (free tier). Use `gemini-1.5-flash` for chat and `text-embedding-004` for embeddings.
- **Ollama (local, optional but encouraged)** — install from `ollama.com`, run a small model such as `llama3.2` or `phi3` and `nomic-embed-text` for embeddings. Great for offline/no-cost.
- Never commit API keys. Keep them in environment variables / Supabase secrets.

RAG stack

- Store embeddings in **Supabase pgvector** (enable the `vector` extension) or a local vector store (Chroma / FAISS).
- Pipeline: chunk → embed → store → retrieve top-k by cosine similarity → feed to the LLM as context.

The "train & show ≥70% accuracy" task

- Take a labelled slice of your seed data, split **train/test (e.g. 80/20)**, build a simple classifier (LLM few-shot classification, or a scikit-learn model on embeddings), and **report accuracy on the held-out test set — target ≥70%**.
- Show the number: a metrics printout / confusion matrix screenshot in your submission.

Generating 5,000+ seed records

- Use a script (Faker / an LLM) to generate ≥5,000 realistic records for your domain.
- Make **3 query sets** to evaluate RAG: (a) **matching** — answerable directly; (b) **extreme/edge** — rare, long, or ambiguous; (c) **noisy** — inject typos, junk tokens and irrelevant rows. Report retrieval quality (hit-rate / precision@k) **with and without noise** and explain the difference.

How you will be scored (Assessment 2)

Your Tech Lead scores you on these parameters — the full point breakdown is in the shared **Assessment 2 Scoring Parameters** sheet. Deliver against each to score well:

Parameter	Max
AI agent — new or meaningfully enhanced (tool-calling / multi-step)	20
Two new workflows / automations, working end-to-end	15
RAG on ≥5,000 records — matching + extreme + noise, quality reported	20
LLM integration (free API / Ollama) + train & show ≥70% accuracy with evidence	15
Test automation — unit tests for all modules + E2E happy paths	15
Best-possible UI/UX	10
Docs, metrics & Dev Control Tower submission	5
Total	**100**
Bonus — Ollama local model working; agent takes real multi-step actions	+10

What to submit on Dev Control Tower (by the deadline)

1. Updated **live URL** (Vercel) and **GitHub repo** link (Assessment 2 branch or tagged release).
2. A short **README / demo notes** covering: the agent, the 2 workflows, the RAG design, and how to run the tests.
3. **RAG evaluation results** — retrieval quality with vs without noise (a small table or screenshot).
4. **Accuracy evidence** — the ≥70% result (metrics printout / confusion matrix), and which LLM/model you used.
5. **Test run output** — unit tests passing + at least one E2E happy-path.
6. Updated **test credentials** for each role (as in Assessment 1).

> **Golden rule:** finish and polish what works before chasing bonus points. A clean, tested, AI-powered app beats a broken pile of half-features.